

Název školy: [Vaše škola]

Název oboru: [Váš obor]

Adresa školy: [Adresa vaší školy]

Dokumentace k projektu

Lunar Lander

Jméno a příjmení: [Vaše jméno]

Název předmětu: [Název předmětu]

Rok vytvoření: 2025

Obsah

1	Cíl práce	2
2	Popis práce	2
2.1	Herní mechaniky	2
2.2	Uživatelské rozhraní a průběh hry	3
2.3	Algoritmy	4
3	Systémové požadavky (System requirements)	4
4	Základní struktura	4
5	Testovací data	6
5.1	Jednotkové testy (Unit Tests)	6
5.2	Manuální testování	6
6	Uživatelská příručka	7
6.1	Hlavní menu	7
6.2	Nastavení	7
6.3	Hra (PlayingState)	8
7	Závěr	8
8	Zdroje	9

1 Cíl práce

Cílem projektu bylo vytvořit 2D počítačovou hru inspirovanou klasikou "Lunar Lander". Hráč ovládá lunární modul s omezeným množstvím paliva a jeho úkolem je bezpečně přistát na určených přistávacích plošinách na povrchu Měsíce. Projekt by měl obsahovat následující klíčové funkce:

- Realistickou (v rámci 2D arkády) fyziku pohybu modulu, zahrnující gravitaci, tah hlavního motoru a rotační trysky.
- Správu paliva, které se spotřebovává používáním motoru.
- Detekci kolizí modulu s terénem a přistávacími plošinami.
- Systém hodnocení kvality přistání (havárie, tvrdé přistání, perfektní přistání) ovlivňující skóre.
- Několik herních stavů: hlavní menu, samotná hra a obrazovka nastavení.
- Možnost upravit hlasitost zvuků a změnit ovládací klávesy.
- Zvukové efekty pro akce ve hře.
- Procedurálně generovaný terén s různě bodovanými přistávacími plošinami.

2 Popis práce

Program implementuje hru Lunar Lander, kde hráč manévruje s kosmickou lodí, snaží se šetřit palivo a bezpečně přistát na Měsíci. Hra je napsána v jazyce Java a využívá standardní knihovny pro grafiku a zvuk.

2.1 Herní mechaniky

- **Ovládání modulu:** Hráč ovládá tah hlavního motoru (proti gravitaci) a dvě rotační trysky pro natáčení modulu doleva a doprava.
- **Fyzika:** Pohyb modulu je ovlivněn konstantní lunární gravitací (`LUNAR_GRAVITY_MS2`). Hlavní motor poskytuje zrychlení proti směru orientace modulu (`MAIN_THRUSTER_ACCELERATION_MS2`). Rotace probíhá konstantní rychlostí (`ROTATION_DEGREES_PER_SECOND`).
- **Palivo:** Modul startuje s maximálním množstvím paliva (`MAX_FUEL`). Používání hlavního motoru spotřebovává palivo (`FUEL_CONSUMPTION_RATE_PER_SECOND_AT_FULL_THRUST`). Pokud palivo dojde, motor přestane fungovat.
- **Terén a přistávací plošiny:** Terén je generován na základě předdefinovaného vzoru (`RAW_TERRAIN_DATA` v `Terrain.java`) a opakuje se, přičemž se na něm náhodně objevují přistávací plošiny s různými násobiteli skóre.
- **Přistání a havárie:** Kontakt s terénem je vyhodnocen na základě kolize sady bodů na modulu (podvozek, špička) s polygonem terénu. Kvalita přistání závisí na vertikální a horizontální rychlosti a úhlu modulu při dopadu. Hranice pro havárii jsou

CRASH_LIMIT_SPEED_Y, CRASH_LIMIT_SPEED_X a CRASH_LIMIT_ANGLE_DEG. Perfektní přistání vyžaduje rychlosti pod PERFECT_LANDING_SPEED_Y, PERFECT_LANDING_SPEED_X a úhel pod PERFECT_LANDING_ANGLE_DEG. Havárie vede k nulovému skóre a penalizaci paliva.

- **Skóre:** Skóre se počítá na základě kvality přistání a násobitele přistávací plošiny. Celkové skóre se kumuluje napříč pokusy, dokud nedojde palivo nebo hráč neukončí hru.
- **Zvuk:** Hra obsahuje zvukové efekty pro kliknutí tlačítek, chod motoru, explozi a smrt hráče (vyčerpání paliva po úspěšném přistání). Hlasitost lze globálně regulovat.
- **Kamera:** Kamera sleduje modul, dynamicky scrolluje a přibližuje/oddaluje podle výšky modulu nad terénem (DISTANCE_TO_ZOOM_IN_THRESHOLD, DISTANCE_TO_ZOOM_OUT_THRESHOLD).

2.2 Uživatelské rozhraní a průběh hry

Hra nabízí grafické uživatelské rozhraní.

- **Hlavní menu:** Zobrazuje název hry "Lunar Lander" a volby:
 - Spustit hru (Enter)
 - Nastavení (S)
 - Ukončit (Esc)
- **Herní obrazovka (PlayingState):** Zobrazuje letící modul nad terénem. V horní části je HUD (Heads-Up Display) s informacemi:
 - SKÓRE: Celkové dosažené skóre.
 - ČAS: Uplynulý čas aktuálního pokusu.
 - PALIVO: Zbývajících množství paliva.
 - VÝŠKA: Aktuální výška modulu nad terénem.
 - HOR.RYCHL: Horizontální rychlost modulu.
 - VER.RYCHL: Vertikální rychlost modulu.

Po přistání nebo havárii se zobrazí zpráva (např. "Perfektní přistání!", "Havárie!") a případné získané body. Hráč může pokračovat (Enter) nebo se vrátit do menu (Esc).

- **Nastavení (SettingsState):** Umožňuje:
 - Nastavení hlasitosti pomocí posuvníku.
 - Změnu ovládacích kláves pro akce: Rotace vlevo, Rotace vpravo, Tah motoru.Návrat do menu se provádí klávesou Esc.
- **Výstup v konzoli:** Během běhu hry se v konzoli mohou objevit výpisy aktuálních hodnot FPS a UPS (např. FPS: 60, UPS: 60). Dále se zde mohou zobrazit chybová hlášení, například při problémech s načítáním zvukových či obrazových souborů (např. Chyba: Zvukový soubor nenalezen - /sounds/engine.wav).

2.3 Algoritmy

- **Pohyb modulu:** Výpočet pozice a rychlosti modulu využívá jednoduchou Eulerovu integraci, kde se v každém časovém kroku (`deltaTime`) aktualizuje rychlost na základě zrychlení (gravitace, tah motoru) a následně pozice na základě nové rychlosti.
- **Generování terénu:** Terén je generován z řetězce `RAW_TERRAIN_DATA` obsahujícího souřadnice bodů. Tyto body jsou transformovány (normalizace, škálování na osu X a Y dle `X_AXIS_SHRINK_FACTOR` a mapování na viewport dle `TERRAIN_VIEWPORT_TOP_Y_RATIO` a `TERRAIN_VIEWPORT_BOTTOM_Y_RATIO`) a vytvářejí základní vzor, který se opakuje. Přistávací plošiny jsou definovány v rámci tohoto vzoru a náhodně se zobrazují s pravděpodobností `PAD_SPAWN_PROBABILITY`.
- **Detekce kolizí:** Modul má definované kolizní body (dva na podvozku, jeden na špičce). V každém kroku se kontroluje, zda některý z těchto bodů (transformovaných do světových souřadnic) leží uvnitř polygonu terénu (`terrainPolygon.contains(worldP)`).
- **Výpočet skóre:** Skóre je odvozeno z kvality přistání, která je funkcí rychlostí (V_x , V_y) a úhlu při dopadu. Tyto hodnoty se porovnávají s prahy pro perfektní přistání a havárii. Výsledná kvalita (0-1) se násobí základním skóre (`MAX_BASE_SCORE_FOR_QUALITY_1`) a násobitelem plošiny.

3 Systémové požadavky (System requirements)

- **Programovací jazyk:** Java.
- **Požadovaná verze JDK:** JDK 23 (dle konfiguračního souboru projektu `misc.xml`, konkrétně `languageLevel="JDK_23"` a `project-jdk-name="openjdk-23"`).
- **Knihovny:**
 - Standardní Java knihovny:
 - * AWT a Swing pro grafické rozhraní a zpracování vstupů (`java.awt`, `javax.swing`).
 - * `javax.sound.sampled` pro přehrávání zvuků.
 - Nejsou použity žádné externí knihovny třetích stran.
- **Operační systém:** Libovolný OS s nainstalovaným kompatibilním JDK (Windows, macOS, Linux).

4 Základní struktura

Program je navržen objektově a rozdělen do několika balíčků a tříd s jasně definovanými zodpovědnostmi.

- **Hlavní třídy a balíčky:**

- `Main.java`: Vstupní bod aplikace, vytváří instanci třídy `Game` a spouští hru.
- Balíček `core`:
 - * `Game.java`: Obsahuje hlavní herní smyčku (`run()`), spravuje časování (FPS/UPS), inicializuje a řídí ostatní hlavní komponenty jako `Window`, `InputHandler` a `StateManager`.
 - * `Window.java`: Zajišťuje vytvoření a správu okna hry (`JFrame`) a kreslí plochy (`Canvas`), včetně buffer strategy pro plynulé vykreslování.
- Balíček `input`:
 - * `InputHandler.java`: Zpracovává vstupy z klávesnice a myši (`KeyListener`, `MouseListener`, `MouseMotionListener`). Spravuje aktuální a předchozí stavy kláves/myši.
 - * `KeyBindings.java`: Mapuje herní akce (`GameAction`) na konkrétní kódy kláves (`KeyEvent`). Umožňuje změnu přiřazení kláves.
 - * `GameAction.java`: Výčtový typ (`enum`) definující možné herní akce (např. `ROTATE_LEFT`, `THRUST`).
- Balíček `states`:
 - * `StateManager.java`: Správce herních stavů. Řídí přechody mezi různými stavy hry (`MenuState`, `PlayingState`, `SettingsState`).
 - * `GameState.java`: Rozhraní definující základní metody pro všechny herní stavy (`init`, `onEnter`, `update`, `render`, `handleInput`, `onExit`).
 - * `MenuState.java`, `PlayingState.java`, `SettingsState.java`: Konkrétní implementace herních stavů.
- Balíček `entities`:
 - * `Lander.java`: Reprezentuje lunární modul. Obsahuje logiku pro fyziku pohybu, spotřebu paliva, detekci kolizí, stavy modulu (let, přistání, havárie) a jeho vykreslení.
 - * `Terrain.java`: Generuje a vykresluje herní terén, včetně přistávacích plošin. Poskytuje kolizní polygon terénu.
 - * `LandingPad.java`: Reprezentuje přistávací plošinu s násobitelem skóre.
- Balíček `audio`:
 - * `AudioManager.java`: Singleton třída pro načítání a přehrávání zvukových efektů. Umožňuje nastavení globální hlasitosti.
- Balíček `ui`:
 - * `Slider.java`: UI komponenta pro posuvník, použitá v nastavení pro regulaci hlasitosti.
- **Komunikace mezi komponentami:**
 - `Game` inicializuje a volá metody `StateManager`, `InputHandler` a `Window`.
 - `StateManager` přepíná aktivní `GameState` a volá jeho metody.
 - Jednotlivé stavy (`MenuState`, `PlayingState`, `SettingsState`) využívají

`InputHandler` pro reakci na uživatelské vstupy a `AudioManager` pro přehrávání zvuků.

- `PlayingState` spravuje instance `Lander` a `Terrain`. `Lander` interaguje s `Terrain` pro detekci kolizí.
- `SettingsState.java` komunikuje s `AudioManager` (změna hlasitosti) a s `KeyBindings` (přes `InputHandler`) pro změnu ovládání.

5 Testovací data

Pro zajištění správné funkčnosti programu je doporučeno kombinovat jednotkové testy (JUnit) s manuálním testováním různých herních scénářů.

5.1 Jednotkové testy (Unit Tests)

Projekt obsahuje sadu JUnit testů pro klíčové komponenty:

- `AudioManagerTest.java`: Testuje správnou inicializaci singletonu, omezování hlasitosti na platné rozsahy (0–100%) a funkčnost zvyšování/snižování hlasitosti.
- `KeyBindingsTest.java`: Ověřuje načtení výchozích klávesových zkratk, nastavení nových kláves, řešení konfliktů při přiřazení již použité klávesy a správné generování textového popisu klávesy.
- `LanderTest.java`: Testuje počáteční stav modulu (pozice, rychlost, palivo), spotřebu paliva při použití motoru a logiku rotace. Obsahuje také poznámky o koncepčních testech pro havárie a výpočet skóre, které by vyžadovaly pokročilejší techniky (mocking) pro izolované testování.
- `SliderTest.java`: Prověřuje počáteční hodnotu posuvníku, omezování hodnot na definovaný rozsah, správné nastavení hodnoty a reakci na simulovaný vstup myši (drag and drop).
- `StateManagerTest.java`: Testuje počáteční stav (žádný aktivní stav), správné přepínání mezi definovanými stavy (`Menu`, `Playing`, `Settings`) a kontroluje, zda jsou stavy správných typů.

5.2 Manuální testování

Manuální testování by se mělo zaměřit na komplexní herní scénáře a interakce:

- **Herní scénáře:**
 - Úspěšné perfektní přistání na různých typech plošin (s různými násobiteli skóre).
 - Úspěšné tvrdé přistání.
 - Havárie způsobená překročením povolené vertikální rychlosti.
 - Havárie způsobená překročením povolené horizontální rychlosti.
 - Havárie způsobená příliš velkým náklonem modulu.

- Havárie nárazem do svahu terénu nebo pádem mimo obrazovku.
- Vyčerpání paliva během letu a následný pád.
- Úspěšné přistání s minimálním nebo žádným zbývajícím palivem (ověření zprávy o konci hry).
- **Ovládání a UI:**
 - Funkčnost výchozích ovládacích kláves (W, A, D).
 - Procházení všemi volbami v hlavním menu (Spustit hru, Nastavení, Ukončit).
 - V obrazovce Nastavení:
 - * Regulace hlasitosti pomocí posuvníku a ověření změny úrovně zvuku.
 - * Změna přiřazení kláves pro všechny herní akce; testovat i přiřazení již obsazené klávesy (řešení konfliktu) a speciálních kláves.
 - * Zrušení změny klávesy pomocí ESC.
 - * Ověření funkčnosti nově nastavených kláves ve hře.
- **HUD a zpětná vazba:**
 - Správnost zobrazovaných údajů na HUD (skóre, čas, palivo, rychlosti, výška).
 - Zobrazení korektních zpráv po skončení pokusu (přistání/havárie) a správný výpočet skóre.
 - Funkčnost zvukových efektů (motor, kliknutí, exploze).
 - Chování kamery (scrollování, přibližování/oddalování).

Důvodem pro testování těchto funkcí je ověření robustnosti herní logiky, správnosti fyzikálního modelu, intuitivnosti ovládání a celkového herního zážitku.

6 Uživatelská příručka

Tato příručka popisuje, jak ovládat hru Lunar Lander.

6.1 Hlavní menu

Po spuštění hry se zobrazí hlavní menu s následujícími možnostmi:

- **Spustit hru:** Stiskněte klávesu ENTER.
- **Nastavení:** Stiskněte klávesu S.
- **Ukončit:** Stiskněte klávesu ESC.

6.2 Nastavení

V obrazovce nastavení můžete upravit:

- **Hlasitost:** Pomocí myši klikněte a táhněte posuvník "Hlasitost" pro úpravu celkové hlasitosti hry.
- **Ovládání:** Klikněte myší na herní akci, jejíž klávesu chcete změnit (např. "Rotace vlevo"). Poté stiskněte novou požadovanou klávesu. Pokud chcete změnu zrušit,

stiskněte **ESC** během čekání na novou klávesu. Výchozí ovládání je:

- W - Tah motoru
- A - Rotace vlevo
- D - Rotace vpravo
- Pro návrat do hlavního menu stiskněte klávesu **ESC**.

6.3 Hra (PlayingState)

- **Ovládání modulu:** Použijte nastavené klávesy pro tah motoru, rotaci vlevo a rotaci vpravo.
- **Cíl:** Bezpečně přistát na rovných přistávacích plošinách. Jemné přistání (nízká rychlost, malý náklon) přináší více bodů. Plošiny mohou mít různé bodové násobitele.
- **HUD (Heads-Up Display):** Sledujte informace v horní části obrazovky:
 - **SKÓRE:** Vaše celkové skóre v aktuální herní seanci.
 - **ČAS:** Čas uplynulý v tomto pokusu o přistání.
 - **PALIVO:** Zbývající palivo. Tah motoru spotřebovává palivo. Pokud palivo dojde, nelze použít motor.
 - **VÝŠKA:** Výška spodní části modulu nad terénem přímo pod ním.
 - **HOR.RYCHL:** Horizontální rychlost modulu.
 - **VER.RYCHL:** Vertikální rychlost modulu.
- **Přistání/Havárie:**
 - Příliš vysoká rychlost nebo velký náklon při kontaktu s terénem způsobí "Havárii!" nebo "Tvrdé přistání!".
 - Úspěšné a jemné přistání je označeno jako "Perfektní přistání!".
- **Po skončení pokusu:**
 - Pokud máte stále palivo, stiskněte **ENTER** pro další pokus (zbývající palivo a skóre se přenáší).
 - Stiskněte **ESC** pro návrat do hlavního menu (aktuální herní seance končí).
 - Pokud palivo došlo, hra končí a je nutné se vrátit do menu klávesou **ESC**.
- Kdykoliv během hry můžete stisknout **ESC** pro okamžitý návrat do hlavního menu (aktuální pokus a seance budou ukončeny).

7 Závěr

Vývoj hry Lunar Lander byl zajímavou zkušeností, která přinesla několik výzev i poučení.

Problémy a výzvy během tvorby:

- **Ladění fyziky:** Nalezení správné rovnováhy mezi gravitací, silou motoru, rychlostí rotace a spotřebou paliva tak, aby byl modul ovladatelný, ale hra stále představovala výzvu, vyžadovalo iterativní testování a úpravy konstant.

- **Detekce kolizí:** Implementace přesné a zároveň výpočetně nenáročné detekce kolizí mezi rotujícím modulem (reprezentovaným několika body) a dynamicky generovaným polygonem terénu.
- **Návrh skórovacího systému:** Vytvoření systému, který spravedlivě odměňuje kvalitu přistání a zohledňuje různé faktory (rychlosti, úhel, násobitel plošiny), bylo klíčové pro motivaci hráče.
- **Kamera a zoom:** Implementace plynulého pohybu kamery, která sleduje modul a zároveň se inteligentně přibližuje/oddaluje v závislosti na výšce nad terénem, byla důležitá pro přehlednost.
- **Generování terénu:** Transformace dat terénu a zajištění plynulého napojování opakujícího se vzoru při pohybu kamery.
- **Jednotkové testování:** Testování částí herní logiky úzce spjatých s vykreslováním nebo komplexními interakcemi objektů (např. `LanderTest` pro kolize) bylo obtížné bez pokročilejších technik (mocking), jak je zmíněno v komentářích testů.

Zavrhnuté implementace (hypoteticky):

- Mohla být zvažována jednodušší reprezentace modulu jako jednoho bodu, ale pro lepší vizuální dojem a přesnější kolize byl zvolen model s definovaným tvarem a více kolizními body.
- Původní systém terénu mohl být čistě náhodný, ale pro lepší kontrolu nad hratelností a možností umístění specifických plošin byl zvolen přístup s opakujícím se vzorem definovaným daty.

Celkový průběh a dojem z práce: Projekt úspěšně implementuje klíčové mechaniky hry Lunar Lander. Nabízí kompletní herní smyčku od hlavního menu, přes samotnou hru, nastavení, až po vyhodnocení pokusů a návrat do menu. Funkce jako kumulativní skóre, přenos paliva mezi pokusy (pokud nedošlo), detailní HUD a možnost konfigurace přispívají k celkovému hernímu zážitku. Kód je strukturován do logických celků (balíčků a tříd), což usnadňuje orientaci a případné další rozšíření. Práce na projektu byla přínosná pro prohloubení znalostí v oblasti objektově orientovaného programování v Javě, základů herní fyziky, správy stavů a návrhu uživatelského rozhraní.

8 Zdroje

Při vývoji tohoto projektu byly primárně využity standardní knihovny a funkce jazyka Java.

1. Oracle. *Java® Platform, Standard Edition & Java Development Kit Version 23 API Specification*. Dostupné online: <https://docs.oracle.com/en/java/javase/23/docs/api/index.html> (Přístup závisí na aktuální dokumentaci JDK 23).
2. Pro obecné principy hry "Lunar Lander" bylo čerpáno z veřejně dostupných informací o této klasické hře.

Žádné externí knihovny třetích stran (mimo standardní JDK) nebyly použity. Konkrétní úryvky kódu nebo specifické algoritmy z externích zdrojů nebyly přebírány bez uvedení, pokud by takový případ nastal.